Buffer Cache in Linux

Operating Systems Case Study: From File Access to Storage

Prashant Jadav

DEPSTAR – CHARUSAT

Learning Objectives

By the end of this case study, students should be able to:

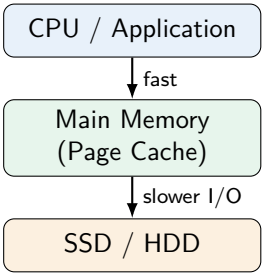
- Explain why operating systems cache file data in main memory.
- Trace a read request through Linux's page cache.
- Explain how buffered writes become persistent storage writes.
- Distinguish the page cache from the older, separate buffer-cache model.
- Explain cache hits, misses, writeback, and cache coherence.

Terminology note

In modern Linux, ordinary buffered file I/O is primarily cached in the **page cache**. "Buffer cache" is still used informally and historically. Linux also uses buffer-head metadata for selected block-oriented operations, but this is not a separate general-purpose cache of all file contents.

Why Does Linux Need a Cache?

- CPU and RAM are much faster than persistent storage.
- Applications repeatedly read common files: libraries, source files, and configuration files.
- Keeping recently used data in RAM avoids repeated storage I/O.
- Writes can be collected and issued later, improving throughput.



Core idea

Use RAM to avoid paying the cost of storage access every time the same file data is requested.

What Is the Linux Page Cache?

Definition

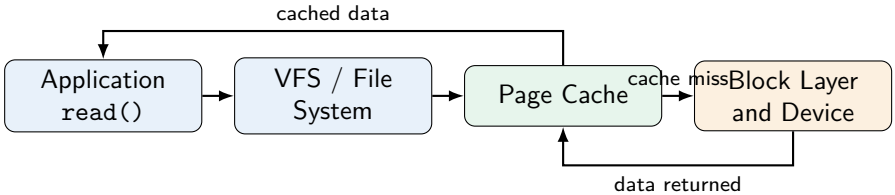
The **page cache** is the kernel-managed cache of file contents held in RAM. It stores file data in memory pages (or folios in current kernel internals), indexed through a file's address-space mapping and file offset.

- Cached contents are associated with the file's inode/address space.
- Multiple processes accessing the same file can often reuse the same cached data.
- Cached data can be clean (matches storage) or dirty (modified in RAM and awaiting writeback).
- The kernel can reclaim cache memory when applications need RAM.

Cache hit vs. miss

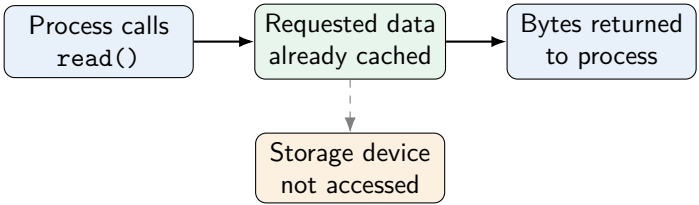
A *hit* means the requested data is already resident and usable in memory. A *miss* means the required data must be brought in, commonly through storage I/O.

Linux File-Read Path: Big Picture



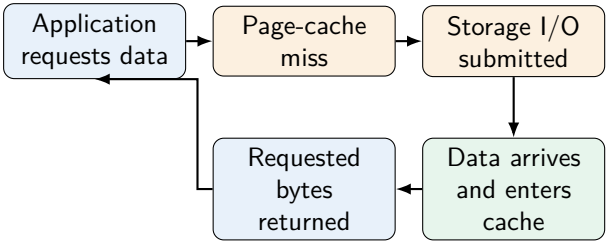
1. The application requests bytes from a file.
2. The VFS and filesystem locate the requested file range.
3. Linux checks whether the corresponding data is in the page cache.
4. On a miss, the kernel requests missing data from storage.
5. The data is cached and then copied or mapped for the application.

Read Operation: Cache Hit



- The kernel finds the requested file range in memory.
- No physical read is required for that cached range.
- System-call, permission-check, copying, and mapping costs may still remain.

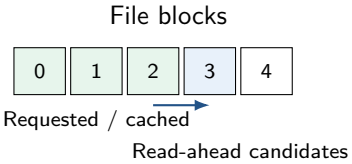
Read Operation: Cache Miss



- Linux initiates I/O for missing file data; nearby data may also be read ahead.
- The process may wait for data, depending on the I/O path and request.
- Once populated, the cached data can serve later reads.

Read-Ahead: Fetching Nearby Data

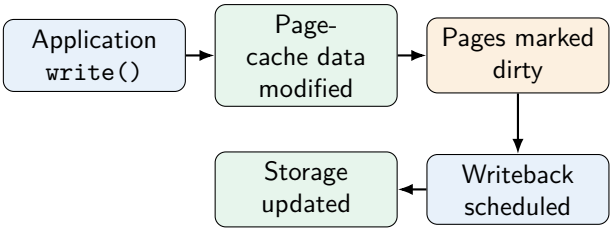
- Sequential access often predicts that upcoming bytes will be needed soon.
- Linux may request additional file ranges before the application asks for them.
- Correct predictions can turn later reads into cache hits.
- Incorrect predictions can waste I/O and memory.



Example

A program reads a large log file sequentially. Fetching upcoming ranges early can reduce stalls between consecutive reads.

Write Operation: Buffered Write



- Ordinary buffered `write()` commonly updates cached file data and marks it dirty.
- The kernel may write dirty data later, in batches.
- A successful `write()` alone does **not** generally guarantee persistence on physical storage.
- Applications needing durability use synchronization such as `fsync()` or `fdatasync()`, subject to filesystem and device guarantees.

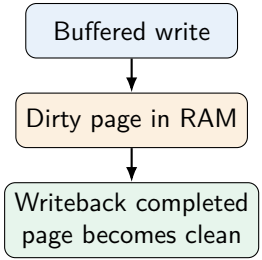
Dirty Pages and Writeback

Clean page

Cached contents match the filesystem's current stored contents. The page can often be discarded if memory is needed.

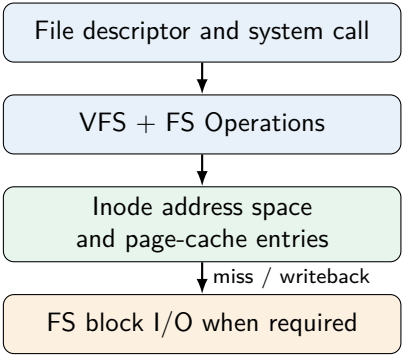
Dirty page

Cached contents have been modified and still need writeback. Changes must be preserved until writeback completes or another safe mechanism handles them.



Writeback can be triggered by background kernel activity, memory pressure, explicit synchronization, or filesystem-specific policies.

Linux Internals: Where Does the Cache Live?



- The VFS provides a common interface across filesystems.
- The filesystem maps file offsets to storage and coordinates cache operations.
- Modern Linux page-cache data is represented through kernel memory-management structures; current kernel internals increasingly use *folios*.
- Buffer heads may describe block-level buffers in selected paths; they are'nt page cache.

Case Study: Reading the Same File Twice

Scenario

A student reads a 20 MiB source file twice. Assume the file fits in available memory and is not evicted between reads.

First read

1. Some ranges are absent from cache.
2. Linux submits storage reads.
3. Data populates the page cache.
4. The application receives the bytes.

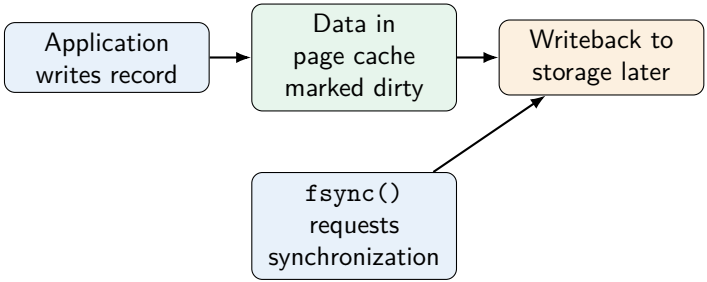
Second read

1. Requested ranges are already cached.
2. Linux reuses in-memory data.
3. Less physical read I/O may be needed.
4. The second read is often faster.

Qualification

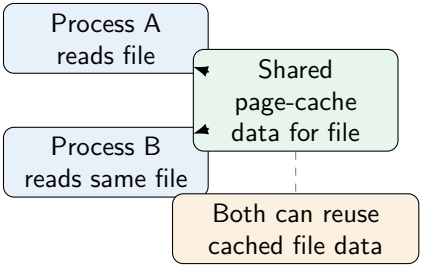
The speedup is not guaranteed: the file may be evicted, the workload may differ, or device and filesystem caches may affect timings.

Case Study: Buffered Write and Durability



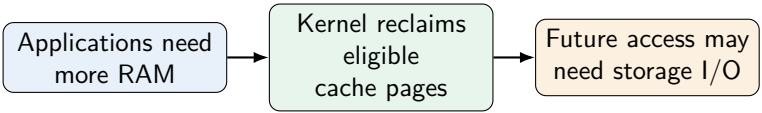
- If power fails before dirty data is safely persisted, recent writes may be lost.
- `fsync(fd)` requests synchronization of file data and required metadata according to filesystem semantics.
- If an application creates or renames a file and needs the directory entry to be durable, directory synchronization may also be required.
- Durability depends on filesystem, device cache, and hardware honoring flush/barrier requests.

Cache Sharing: Two Processes, One File



- Processes accessing the same file normally share the kernel's page-cache data for that file.
- This avoids maintaining an independent file-content cache for every process.
- Memory-mapped file access also interacts with the page cache.
- Concurrent modification and visibility rules depend on access method and application synchronization.

Memory Pressure and Cache Reclamation



- Clean cached pages can generally be discarded and read again later.
- Dirty pages usually need writeback before their memory can be safely reclaimed.
- Linux balances cache usefulness against application memory requirements.
- A large page cache is not automatically a problem: reclaimable cache memory can be reused.

Observing Cache Activity on Linux

```
# Overall memory and cache overview
free -h

# Selected kernel memory counters
grep -E '^(MemAvailable|Cached|Buffers|Dirty|Writeback):' /proc/meminfo

# Per-process I/O counters (interpret cautiously)
cat /proc/<PID>/io

# Read a file twice and compare elapsed time
/usr/bin/time -f '%e seconds' cat large-file.bin > /dev/null
/usr/bin/time -f '%e seconds' cat large-file.bin > /dev/null
```

Measurement caution

Command timing includes process startup and other overhead. The first run is not guaranteed to be a cold-cache run. Do not drop system caches on a shared or production machine.

Common Misconceptions

- **“A successful `write()` means data is on disk.”**
  Not generally true for buffered I/O; dirty data may remain in RAM.
- **“The page cache is only for reads.”**
  No. Buffered writes commonly modify cached data and mark it dirty.
- **“Linux has two separate full caches for all file contents.”**
  This is an outdated simplification; the page cache is the primary file-content cache.
- **“Cached memory is permanently unavailable to applications.”**
  Much of the page cache is reclaimable under memory pressure.
- **“A cache hit makes the read free.”**
  It avoids storage I/O for resident data, but other costs remain.

Summary

1. Modern Linux primarily uses the **page cache** for ordinary buffered file contents.
2. A read hit reuses cached data; a miss fetches missing data from storage.
3. Read-ahead can fetch likely-needed ranges before an explicit request.
4. Buffered writes commonly mark cached pages dirty; writeback persists them later.
5. `fsync()` and related interfaces matter when an application needs stronger durability.
6. The kernel reclaims eligible cache pages under memory pressure.

Takeaway

The cache is a performance layer between file operations and persistent storage. It improves reuse and write efficiency, while writeback and synchronization govern persistence.

Discussion Questions

1. Why can the second read of a file be faster than the first?
2. Draw the path taken by a read request on a cache hit and a cache miss.
3. What is the difference between a clean page and a dirty page?
4. Why does a successful buffered `write()` not necessarily guarantee durability?
5. How can read-ahead improve performance, and when can it be wasteful?
6. Why is “page cache” more precise than “buffer cache” for modern Linux file contents?

References



Linux kernel documentation, *Memory Management*.
`https://docs.kernel.org/admin-guide/mm/`



Linux kernel documentation, *Overview of the Linux Virtual File System*.
`https://docs.kernel.org/filesystems/vfs.html`



Linux man-pages project, *read(2)*, *write(2)*, and *fsync(2)*.
`https://man7.org/linux/man-pages/`



Robert Love, *Linux Kernel Development*, 3rd edition, Addison-Wesley.

Kernel implementation details evolve; consult documentation for the target kernel when exact internals matter.